

Lab 1 Exam

准备工作：创建并切换到 lab1-exam 分支

请在自动初始化分支后，在开发机依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab1-exam
$ cd tools/readelf64
```

请在 tools/readelf64 目录下完成本题的代码实现。

题目背景

符号表（.symtab 节）是 ELF 文件中的一个重要节，记录了程序中的符号信息（如函数名、变量名等等），在链接和调试等过程中都有很重要的作用。因此，本题要求你实现一个简化版的 readelf64，输出 ELF 文件中所有节头的相关信息，并找出符号表（.symtab 节）对应节头在节头表中的下标。

课下练习中，我们已经实现过一个简化版 readelf，用于解析 32-bit little-endian ELF 文件并输出节头地址。本题在此基础上进一步扩展。出于实用性考虑，要求同学们解析 64-bit little-endian ELF 文件。

需要注意的是，64 位 ELF 的整体解析思路与 32 位版本基本一致，主要差别在于所使用的数据结构和部分字段类型不同。更具体地说，本题中最直接的变化是：原先使用的 Elf32_ 系列结构体/变量，需要相应替换为 Elf64_ 系列结构体/变量。其余区别同学们在本题中无需考虑。

在 ELF 文件中，节头表中的每一个表项都描述了一个节（section）的信息，包括该节在文件中的偏移、大小、类型等。每个节头表表项都对应一个 Elf64_Shdr 结构体。通过解析节头表，我们可以获得 ELF 文件中所有节的基本信息。

本题要求你实现一个简化版的 readelf64，输出：

1. 该 ELF 文件中的 **总节数（即节头表表项数量）**；
2. 符号表（.symtab 节）对应节头 **在节头表中的下标**；
3. 每一节的 **名称（后文讲解获取方法）、文件偏移（sh_offset）和大小（sh_size）**。

相关知识说明

A. ELF64 文件头（Elf64_Ehdr *ehdr，与 32 位版本相对应）

ELF 文件开头为 **ELF 文件头（ELF Header）**，它描述了这个 ELF 文件的整体布局信息。

它的作用是：告诉我们后续该去哪里找 **节头表、节名称字符串表** 等关键信息。

在本题中，你只需要 **重点关注该结构体的以下字段**：

- e_shoff：节头表在 **文件中的偏移**；
- e_shnum：节头表表项的数量；
- e_shstrndx：**节名称字符串表** 的索引（即，其对应节头在节头表中的下标，**相关知识说明 C** 中会展开讲解）

对应到实现中，可以通过 `e_shoff` 和 `e_shnum` 拿到与遍历节头表，用 `e_shstrndx` 定位 **节名称字符串表**。

B. 节头表与节头结构体 (`Elf64_Shdr *sh_table`，与 32 位版本相对应)

节头表 (Section Header Table，代码中对应 `sh_table`)，可以理解为“全体节的目录”。

在本题中，可以直接将节头表视为由 `Elf64_Shdr` 构成的数组，其作用是：按顺序记录 ELF 文件中每一个节 (section) 的关键信息，便于程序遍历和查询。

节头表里的每一个表项都对应一个 `Elf64_Shdr` 结构体 (也称为节头)。若将节头表 `sh_table` 视为数组，则节头包括：`sh_table[0]`，`sh_table[1]`，...，`sh_table[e_shnum-1]`。也就是说，每个 `Elf64_Shdr` 结构体描述一个节的信息。

它们的作用是告诉我们每个节的名字、内容在文件中的位置、大小等信息。

本题 **重点关注该结构体的以下字段**：

- `sh_name`：节名称在 **节名称字符串表内容字符串** 中的 **偏移**；
- `sh_offset`：该节在 **文件中的偏移**；
- `sh_size`：该节的大小。

注意：`sh_name` 不是字符串指针，而是一个 **偏移量**。要想获得节名，需要先找到 **节名称字符串表内容字符串**。

对应到实现时，你需要遍历所有 `Elf64_Shdr`，利用 `sh_name` 解析节名，而后输出节名及该节的 `sh_offset` 和 `sh_size`。

C. 节名称字符串表

节名称字符串表 (通常为 `.shstrtab` 节)，本身也是 ELF 文件中的一个节。

它的内容本质上是一个由 `\0` 分隔的字符串池，存放所有节名。

例如，其内容字符串可能类似 (不含双引号 `"`)：

```
"\0.text\0.data\0.bss\0.symtab\0.strtab\0.shstrtab\0..."
```

它的作用是：配合每个节头里的 `sh_name` 偏移，解析出真实节名。

以上述节名称字符串表为例，若某个节头的 `sh_name` 为 `1`，则该节的名称为 `.text`；若 `sh_name` 为 `13`，则该节的名称为 `.bss`。

那么如何找到节名称字符串表，又如何借此获取某一节的节名称呢？

注意，ELF 文件头中的 `e_shstrndx` 表示 **节名称字符串表** 对应 **节头** 在 **节头表** 中的下标。

由此，给出获取某一节 **节名称** 的过程：

1. 根据 `e_shoff` 找到节头表 (`Elf64_Shdr *sh_table = ((char *)binary + ehdr->e_shoff)`)；
2. 取出下标为 `e_shstrndx` 的那个节头 (`Elf64_Shdr *shstrShdr = &sh_table[ehdr->e_shstrndx]`)；

3. 通过该节头的 `sh_offset`（内容在文件中的偏移），找到节名称字符串表的内容字符串的位置，存到 `char *shstrtab`（请大家参考 1. 中 `e_shoff` 的用法，完成相关代码）；
4. 对于某一节（若其对应节头为 `shdr`），其节名可由 `char *sectionName = shstrtab + shdr->sh_name` 得到。

题目要求

本题要求你阅读 `tools/readelf64` 目录下文件，并补全代码。

重点查看以下文件：

- `elf64_mini.h`：给出了本题涉及的若干 ELF64 相关结构体/变量的定义供参考；
- `readelf64.c`：本题需要补全的核心代码。

你需要在 `tools/readelf64/readelf64.c` 中补全代码，完成以下功能：

1. 读取 **ELF64 文件头**；
2. 定位 **节头表**；
3. 通过 **节名称字符串表** 解析 **节头的名称**；
4. 遍历所有节头，输出每一节的：
 - **序号**（从 0 开始）
 - **节名称**（`section_name`）
 - **文件偏移** `sh_offset`
 - **大小** `sh_size`
5. 在遍历过程中，找到名字为 `.symtab` 的节，并输出它在**节头表中的序号**（从 0 开始计算）。

输出格式

要求你的程序输出格式如下：

```
section_count=<count>
[0] name="<section_name>"   offset=0x<offset>   size=<size>
[1] name="<section_name>"   offset=0x<offset>   size=<size>
...
[<count>-1] name="<section_name>"   offset=0x<offset>   size=<size>
symtab_index=<index>
```

其中：

- 第一行 `<count>` 为节头总数，即节头表表项的数量；
- 最后一行 `<index>` 为名称恰好等于 `.symtab` 的节头序号；
- 从第二行起，`[i]` 表示第 `i` 个节头的信息；
- `<offset>` 为该节的 `sh_offset`，以小写十六进制输出，带 `0x` 前缀，不补前导零；
- `<size>` 为该节的 `sh_size`，按十进制输出；
- 即使某个节名称为空字符串，也应正常输出。

输出节头信息的格式串请严格使用：

```
"[%d]\tname=\"%s\" \toffset=0x%lx\tsize=%lu\n"
```

样例输出 & 本地测试

你可以在本地通过如下方式测试：

```
$ cd tools/readelf64
$ make
$ ./readelf64 hello
```

其输出应为：

```
section_count=39
[0] name="" offset=0x0 size=0
[1] name=".interp" offset=0x318 size=28
[2] name=".note.gnu.property" offset=0x338 size=48
[3] name=".note.gnu.build-id" offset=0x368 size=36
...
[36] name=".symtab" offset=0x35b0 size=840
[37] name=".strtab" offset=0x38f8 size=436
[38] name=".shstrtab" offset=0x3aac size=399
symtab_index=36
```

说明：

- 完整的样例输出已下发到 `tools/readelf64/hello.txt` 文件中，供同学们参考。
- 评测时将根据需求使用不同的 ELF 文件进行测试，请不要依赖样例输出中的具体数值。

提交评测

请在开发机中执行下列命令后，在课程网站上提交评测：

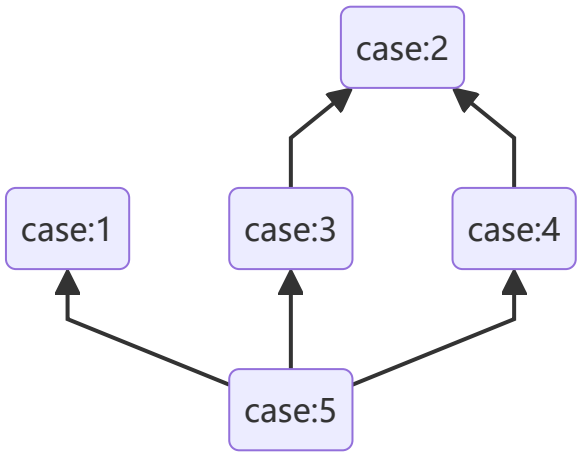
```
$ cd ~/学号/
$ git add -A
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push
```

评测标准

测试点序号	评测内容	分值
1	恰为样例	10 分
2	正确读取 ELF64 文件头，并输出 <code>section_count</code>	20 分
3	正确输出所有节头的 <code>name</code> 、 <code>offset</code> 和 <code>size</code>	40 分
4	正确找到 <code>.symtab</code> 输出 <code>symtab_index</code>	20 分

测试点序号	评测内容	分值
5	所有输出内容完全正确	10 分

测试点依赖关系如下：



测试用例补充说明

- 所有测试文件均为 **64-bit little-endian ELF 文件**；
- 本题只涉及 **ELF 文件头**、**节头表** 和 **节名称字符串表**；
- 本题不要求解析程序头表，也不要求解析符号表项内容；
- 在线评测时，`tools/readelf64` 目录下的 `elf64_mini.h`、`hello`、`hello.txt`、`main.c`、`Makefile` 都可能被替换为标准版本，因此请不要在这些文件中编写实际功能所依赖的代码，也不建议对它们进行任何修改，以免丢失样例环境；
- `elf64_mini.h` 仅保留了本题所需的最小结构体定义，请以此为主要参考。

补充提示

- **强烈建议** 通过 **回顾指导书/切换分支/复制文件** 等方法，参考课下练习中 **32 位版本 readelf** 的实现思路来完成本题任务。
- 本题中所有 **“序号”“索引”“下标”** 等关键词，均从 **0** 开始记。
- **字符串比较** 可以使用 `string.h` 中的 `strcmp`，函数原型为：

```
int strcmp(const char *s1, const char *s2);
```

当返回值为 **0** 时，表示两个字符串相同；否则即两个字符串有差异。

- **ELF 文件结构示意图（建议仔细参考）**

```
File Offset
(base)

base + 0x0000
+-----+
| ELF Header |
| e_shoff ---> 节头表起始偏移 |
```

```

| e_shnum ---> 节头表项数量 |
| e_shstrndx ---> 节名称字符串表对应节头下标 x |
+-----+

base + e_shoff
+-----+
| Section Header Table |
+-----+
| [0] |
+-----+
| [1] |
+-----+
| [2] |
+-----+
| ... |
+-----+
| [x] 节名称字符串表 .shstrtab 对应节头 |
| sh_name ---> 节名称在节名称字符串表中的偏移 |
| sh_offset ---> 节内容的文件偏移 |
| sh_size ---> 节大小 |
| ... |
+-----+
| [y] 符号表 .symtab 对应节头 |
| sh_name ---> 节名称在节名称字符串表中的偏移 |
| sh_offset ---> 节内容的文件偏移 |
| sh_size ---> 节大小 |
| ... |
+-----+

base + sh_table[x].sh_offset
+-----+
| 节名称字符串表内容 |
| "\0.text\0.data\0.bss\0.symtab\0.strtab\0.shstrtab\0..." |
| ↑ 各节头的 sh_name 都是在这张表中取字符串 |
+-----+

```

- 若评测机反馈 运行时遇到错误 `exit-code=139`，请检查是否存在 **非法内存访问**（如访问了不属于该 ELF 文件的内存地址）。你可以通过 `gdb` 等工具进行调试，定位问题所在。